

The “8 DevOps steps” describe the continuous software delivery lifecycle used by modern engineering teams. These stages work together in a loop so teams can build, ship, and improve software faster and more reliably.

1. Plan

Planning defines **what needs to be built** and how the work will be organized.

Main goals

- Gather requirements
- Define features and priorities
- Break work into tasks
- Estimate timelines
- Coordinate teams

Typical activities

- Sprint planning
- User stories
- Backlog management
- Architecture discussions
- Risk analysis

Common tools

- [Jira](#)
- [Trello](#)
- [Azure DevOps](#)
- [Notion](#)

Example

A company plans an e-commerce “wishlist” feature:

- Users can save products
- Product managers define requirements
- Developers estimate effort
- Tasks are assigned for frontend, backend, and testing

2. Code

Developers write and manage application source code.

Main goals

- Develop features
- Fix bugs
- Maintain code quality
- Collaborate safely

Typical activities

- Writing code
- Code reviews
- Branching and merging
- Version control

Common tools

- [Git](#)
- [GitHub](#)
- [GitLab](#)
- [Bitbucket](#)

Example

Developers create:

- Backend APIs
- Database logic
- UI components
- Authentication code

Important DevOps practice

Code is committed frequently into a shared repository.

3. Build

The build stage converts source code into a runnable application.

Main goals

- Compile code
- Package applications
- Resolve dependencies
- Generate artifacts

Typical activities

- Compile Java/C++ code
- Build Docker images
- Package binaries
- Install dependencies

Common tools

- Maven
- Gradle
- Docker
- Webpack

Example

A Node.js app:

- Installs npm packages
- Creates production bundle
- Builds Docker container image

CI connection

This step is usually automated through Continuous Integration (CI).

4. Test

Testing ensures the application works correctly before release.

Main goals

- Detect bugs early
- Validate features
- Improve reliability
- Prevent regressions

Types of testing

- Unit testing
- Integration testing
- API testing
- UI testing
- Security testing
- Performance testing

Common tools

- JUnit
- Selenium
- Postman
- Cypress

Example

Before deployment:

- APIs are tested automatically
- Login flow is validated
- Security scans check vulnerabilities

DevOps principle

Testing should be automated whenever possible.

5. Release

The release stage prepares software for production delivery.

Main goals

- Approve stable builds
- Manage versions
- Prepare release notes
- Coordinate deployment readiness

Typical activities

- Tagging versions

- Approvals
- Artifact storage
- Change management

Common tools

- [Nexus Repository](#)
- [JFrog Artifactory](#)
- [GitHub Releases](#)

Example

Version `v2.1.0` is approved after:

- All tests pass
- QA approval
- Security validation
- Documentation updates

6. Deploy

Deployment pushes the application into environments such as staging or production.

Main goals

- Deliver software safely
- Reduce downtime
- Automate infrastructure
- Enable fast rollbacks

Types of deployment

- Manual deployment
- Automated deployment
- Blue-green deployment
- Canary deployment
- Rolling deployment

Common tools

- [Kubernetes](#)
- [Jenkins](#)

- [Argo CD](#)
- [Ansible](#)

Example

Deployment pipeline:

1. Deploy to staging
2. Run smoke tests
3. Deploy to production automatically

DevOps focus

Deployments should be repeatable and automated.

7. Operate

Operate means maintaining and managing the live production system.

Main goals

- Keep services running
- Manage infrastructure
- Handle incidents
- Scale systems

Typical activities

- Server management
- Kubernetes cluster operations
- Backup handling
- Incident response
- Capacity scaling

Common tools

- [Kubernetes](#)
- [Terraform](#)
- [AWS](#)
- [Azure](#)

Example

Operations team:

- Monitors server load
- Adds new containers during traffic spikes
- Handles outages

Key concept

Infrastructure as Code (IaC) is heavily used here.

8. Monitor

Monitoring tracks system health and user experience continuously.

Main goals

- Detect issues quickly
- Measure performance
- Improve reliability
- Gather business insights

What is monitored

- CPU usage
- Memory usage
- Application errors
- API latency
- User activity
- Security events

Common tools

- [Prometheus](#)
- [Grafana](#)
- [Datadog](#)
- [New Relic](#)

Example

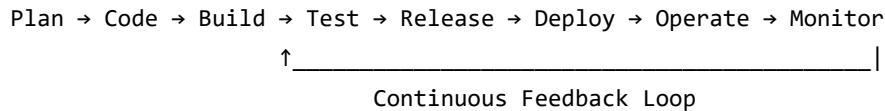
Monitoring detects:

- High response time
- Database slowdown

- Failed login spikes

Alerts are automatically sent to engineers.

Complete DevOps Lifecycle Flow



The monitoring feedback helps teams improve future planning and development.

Real-World Example (Simple)

Imagine building a food delivery app:

Step	Example
Plan	Decide to add live order tracking
Code	Developers write tracking feature
Build	App and backend packages are created
Test	Tracking accuracy is tested
Release	Version 3.0 approved
Deploy	New version goes live
Operate	Servers handle customer traffic
Monitor	Engineers track crashes and delays

Key DevOps Principles Behind These Steps

Automation

Reduce manual work using CI/CD pipelines.

Collaboration

Developers and operations teams work together.

Continuous Improvement

Feedback from monitoring improves future releases.

Fast Delivery

Smaller, frequent releases reduce risk.

Popular CI/CD Pipeline Example

A common DevOps flow:

```
Developer pushes code
↓
CI server builds app
↓
Automated tests run
↓
Artifact released
↓
Application deployed
↓
Monitoring tracks production
```